

# HBase Notes

---



Giovanni  
Brunetti



HBase è un **DB NOSQL** costruito su **HDFS**, è **colonnare**. Basato su Google Big Table paper, i dati vengono recuperati, aggiornati o batch process

**Perché?** Salvare grandi dati (**TB/PS**), capace di gestire **4 mila richieste**, salvare dati **con schemi variabili** o con schemi **variabili**, **random read and write**

**Perché NO?** Se non è adatto DB, si leggono direttamente determinati file, scrivono una volta sola o sempre in append, non ci sono modelli, l'accesso è lento

## HBase Tables

I dati vengono salvati in tabelle solo **funzione di row**, ognuna per una colonna diversa senza uno schema preciso.

Le tabelle sono raggruppate in **NAMESPACE**, usati per raggruppare tabelle per utente o applicazione.

Ogni riga è identificata da una **ROW KEY**, le row sono **ordinate lessicograficamente** tramite la key, **binariamente**

Anche le colonne sono raggruppate in **COLUMN FAMILIES** <sup>(CF)</sup>, che **impongono** anche la **disposizione fisica** dei dati. Per questo motivo devono essere definiti in anticipo e non sono facilmente modificabili. Ogni riga ha le stesse CF, anche se sono vuote.

Le CF sono identificate dalla **COLUMN QUALIFIER** <sup>(CQ)</sup>, una colonna specificata all'inizio per ogni CF.

Una combinazione **ROW KEY-CF-CQ** identifica una **CELLA**. HBase consente più valori per ogni cella, con la **stampa di valori**

HBase Cell Versions dunque permette di inserire valori diversi associati ad un **TIMESTAMP**, questo timestamp viene usato anche per il **TIME-TO-LIVE**

## HBase Architecture

I dati sono quindi organizzati in tabelle, **partizionate** in **REGIONS**, che sono **intervalli continui di ROW KEYS**. Questi vengono hostati da nodi del **CLUSTER** responsabili di esse dette **REGION SERVERS**. Esse sono gestite dal **MASTER node**, gli utenti si collegano direttamente alla **region servers** tramite **RPC layer**. Tutti sono coordinati da **ZOOKEEPER**. Vi sono altri server quali **REST** (per le API) e

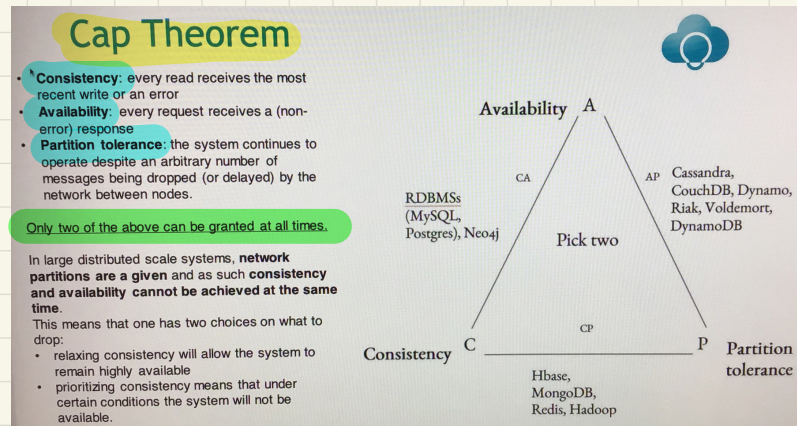
## THRIFT SERVER

## WRITE OPERATION

Le operazioni vengono eseguite **direttamente** su **region servers**. Quella di riferimento viene **identificata** da **Zookeeper** che interroga **.META table**, la quale contiene il **mapping tra region e RS**. La **Region Server** salva i dati in **Hfile**, salvati in seguito su **HDFS**. I cambiamenti ai dati vengono memorizzati nel **HLog** su **HDFS**, che agisce come **WAL** (write ahead log) per recuperare i dati in caso di errore. Infine una cella viene scritta nel **MEMSTORES**, dei buffer di memoria che periodicamente creano nuovi **Hfiles**.

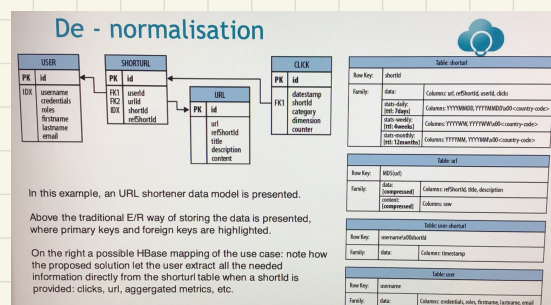
## READ OPERATIONS

Le richieste vengono inviate a **Zookeeper** che indirizza la **RegionServer** con il **.META.Table** che indirizza la regione di riferimento che contiene la **rk** di interesse. Nelle fasi di lettura i **MEMSTORE** e gli **HFiles** vengono uniti on-the-fly per avere un quadro completo della region. E' cruciale la **CACHE** per valorizzare queste operazioni. Per filtrare i file correlati in ogni Regione si usano i filtri Bloom. In caso di failure di un region server, i dati contenuti nel memstore vengono persi, ma il contenuto può essere recuperato dal **HLog**.

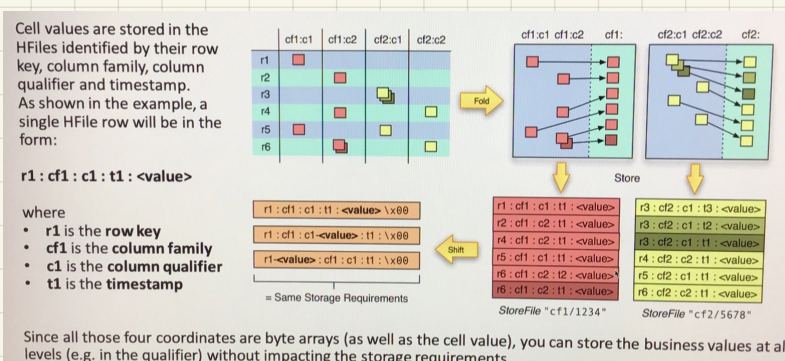


## DENORMALISATION

Spesso bisogna ripensare come i dati vengono archiviati in sistemi NoSQL per **utilizzarli in modo dinamico**. Si usa quindi **DDI** (**Denormalization, Duplication, Intelligent keys**)



## STORAGE LAYER REPRESENTATION



## Differenze con RDBMS e vantaggi

- Gli **RDBMS** si basano su relazioni e normalizzazioni, **HBase** su **denormalizzazione** e **pattern di accesso**.
- **HBase** fornisce un **accesso key-based** ad una cella o ad una insieme sequenziale di celle.
- Le operazioni sono **fornite** su una riga.
- E' **colonnare**, le colonne vengono raggruppate in famiglie e all'interno di esse sono memorizzate solo forma di row.

## ACCEDERE HBASE

- Si può accedere tramite **JAVA API**, **REST (HAP access)**, **THRIFT GATEWAY** (permette accessi da lingue diverse).
- Vi sono **interfacce SQL non Native**: **Phoenix**, **Impala**, **Presto**, **Hive**.

### HBase Client API

#### List of basic operations

| Type      | Kind     | Description                                                                                  |
|-----------|----------|----------------------------------------------------------------------------------------------|
| Get       | Query    | Retrieve previously stored data from a single row; can get entire row or only some cf/cells. |
| Scan      | Query    | Iterate over all or specific rows and return their data; some filters can be used.           |
| Put       | Mutation | Create or update one or more cells in a single row.                                          |
| Delete    | Mutation | Remove a specific cell, column, row, etc.                                                    |
| Increment | Mutation | Treat a column as a counter and increment its value.                                         |
| Append    | Mutation | Attach the given data to one or more columns in a single row.                                |

### Delete

- Deletes work by creating **tombstone markers**. Tombstone markers are just labels with a timestamp assigned to cell versions.
- Only in the **major compaction**, the tombstones are processed to actually **remove the dead values**.
- When HBase does a major compaction, the tombstones are processed to actually remove the deleted cell values, together with the tombstones themselves. If the **version** you specified when deleting a row is larger than the version of any value in the row, then you can consider the complete row to be deleted.

## INTERNAL TABLE OPERATIONS

**MINOR COMPACTION** = compattazione solo alcuni file senza rimuovere le celle eliminate, piccoli HFiles vengono uniti.

**MAJOR COMPACTION** = Tutti i file sono letti come un'unica unità e le celle vengono rimosse. Tutti gli HFiles di una regione vengono riscritti in un unico HFile. Molto onerosa e pesante.

**SPLITS** = Quando una regione supera la memoria limite o è troppo ridotta viene **splitata** in 2.

**BALANCING** = Il **Master** si occupa di fare in modo che ogni **RegionServer** gestisce un numero bilanciato di Regioni.

# HBase Processes

Master → table DDL operations

Region Servers → operations READ/Writes

Region Server con META table → mantiene le regioni con mappa region → region server

Sistemi esterni → Zookeeper → coordinate

Un HBase coprocessor è un codice custom scritto dall'utente.

Endpoint Coprocessor → Simile ad una Stored Procedure, deve essere invocata tramite Protobuf spesso.

Observer Coprocessor → Simile ai trigger. Si attiva ad una azione, disponibile <sup>4 de</sup> per HBase action.

## Observer Coprocessor – Possible Triggers

- Region Server Observer
  - preStopRegionServer
  - preExecuteProcedures
  - postExecuteProcedures
  - preClearCompactionQueues
  - postClearCompactionQueues
- Region Observer
  - preGetOp
  - postGetOp
  - prePut
  - postPut
- WAL Observer
  - preWALRoll
  - preWALWrite
  - postWALRoll
  - postWALWrite
- Master Observer
  - Runs in HBase master
  - Create, Delete, Modify table
  - Clone, Restore, Delete Snapshots
  - Region splits and many more

## Filters

- Similar to Observer Coprocessors for get/scan
- Can filter on any part of a row
- Many built-in filters available
- Can deploy custom filters
- Support for push-down predicates
- Different API
- Came earlier and much more limited

Perché usarli?

Per controllare il comportamento di Region Server o di operazioni, filtri o aggregazioni, ridurre la pressione lato client per analisi di dati non complessi (Phoenix)

## DYNAMIC VS STATIC

| Description               | Static Loading   | Dynamic Loading |
|---------------------------|------------------|-----------------|
| Changes to hbase-site.xml | Yes              | No              |
| Restart region servers    | Yes              | No              |
| Jar location              | Local filesystem | HDFS            |
| Coprocessor availability  | Global           | Per table       |
| Loaded via hbase shell    | No               | Yes             |
| Loaded via java API       | No               | Yes             |
| Read permissions          | HBase            | HBase           |
| Management complexity     | High             | Low             |

## Deployment Guidelines

- Enable user coprocessors
  - Set `hbase.coprocessor.user.enabled` to true
  - Phoenix coprocessors are treated as user coprocessor
- Set `hbase.coprocessor.enabled` to true
  - Keeps system coprocessors enabled (AccessController)
- Enable coprocessor white listing (HBASE-16700)
  - Set `hbase.coprocessor.region.whitelist.paths`
  - Specify each directory individually
  - Wildcards won't include subdirectories (documentation says otherwise)
  - Entire filesystem, i.e., `hdfs://Test-Laptop`, won't work (documentation says otherwise)

## RECAP

- Coprocessors are necessary
  - Phoenix
  - HBase security
- User coprocessors are dangerous
  - Write defensive code
  - Be careful with deployment
- Make use of HBASE-16700
- Cleanup can be messy
  - HBASE-14190 – Assign system tables ahead of user region assignment

## Schema Design

This article covered the basics of HBase schema design. I started with a description of the data model and went on to discuss some of the factors to think about while designing HBase tables. There is much more to explore and learn in HBase table design which can be built on top of these fundamentals. The key takeaways from this article are:

- ◆ Row keys are the single most important aspect of an HBase table design and determine how your application will interact with the HBase tables. They also affect the performance you can extract out of HBase.
- ◆ HBase tables are flexible, and you can store anything in the form of `byte[]`.
- ◆ Store everything with similar access patterns in the same column family.
- ◆ Indexing is only done for the `Keys`. Use this to your advantage.
- ◆ Tall tables can potentially allow you faster and simpler operations, but you trade off atomicity. Wide tables, where each row has lots of columns, allow for atomicity at the row level.
- ◆ Think how you can accomplish your access patterns in single API calls rather than multiple API calls. HBase does not have cross-row transactions, and you want to avoid building that logic in your client code.
- ◆ Hashing allows for fixed length keys and better distribution but takes away the ordering implied by using strings as keys.
- ◆ Column qualifiers can be used to store data, just like the cells themselves.
- ◆ The length of the column qualifiers impact the storage footprint since you can put data in them. Length also affects the disk and network I/O cost when the data is accessed. Be concise.
- ◆ The length of the column family name impacts the size of data sent over the wire to the client (in Key/Value objects). Be concise.